

Lab 8: Raft Consensus Algorithm using Docker

1. Aim

To implement a three-node distributed system using the Raft consensus algorithm inside Docker containers and observe leader election, follower nodes, and fault tolerance.

2. Learning Objectives

After completing this experiment students will be able to:

- Understand distributed systems
- Understand leader election in Raft
 - Use Docker to simulate distributed machines
 - Observe fault tolerance when a leader fails
- Understand container networking

3. Software Requirements

Students must install the following software:

Docker Desktop Python (inside container) Command Prompt / Terminal

To verify Docker installation run:

docker --version

4. Create Project Folder

Step 1

Open File Explorer

Go to Desktop

Create a new folder named:

raft_docker_lab

5. Open Terminal

Open Command Prompt

Navigate to the project folder

cd %USERPROFILE%\OneDrive\Desktop\raft_docker_lab

For linux user:

cd ~/Desktop/raft_docker_lab

6. Create Required Files

Students must create four files inside the folder.

Files:

node.py Dockerfile requirements.txt docker-compose.yml

Inside the terminal run this

type nul > node.py & type nul > Dockerfile & type nul > requirements.txt & type nul > docker-compose.yml

Then check:

dir

**For linux user :touch node.py Dockerfile requirements.txt
docker-compose.yml**

Then check:

ls

Folder Structure

raft_docker_lab

|

|— node.py

|— Dockerfile

|— docker-compose.yml

|— requirements.txt

Take Screenshot 1: Folder structure

```
C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>dir
Volume in drive C is Windows
Volume Serial Number is F23A-D83D

Directory of C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab

16-03-2026  22:09    <DIR>          .
16-03-2026  22:07    <DIR>          ..
16-03-2026  22:09                0 docker-compose.yml
16-03-2026  22:09                0 Dockerfile
16-03-2026  22:09                0 node.py
16-03-2026  22:09                0 requirements.txt
                4 File(s)                0 bytes
                2 Dir(s)  343,472,115,712 bytes free

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>
```

7. Create requirements.txt

Command: **notepad requirements.txt**

For linux user : nano requirements.txt

Paste the following

raftos

Save as

requirements.txt

8. Create Dockerfile

Command: **notepad Dockerfile**

For linux user : nano Dockerfile

Write the code

Save as:

Dockerfile

9. Create docker-compose.yml

Command: **notepad docker-compose.yml**

For linux user : nano docker-compose.yml

Write the code

10. Create node.py

Command: **notepad node.py**

For linux user : nano node.py

Write the code

Save as: **node.py**

11. Build and Run Containers

Open terminal

Run:

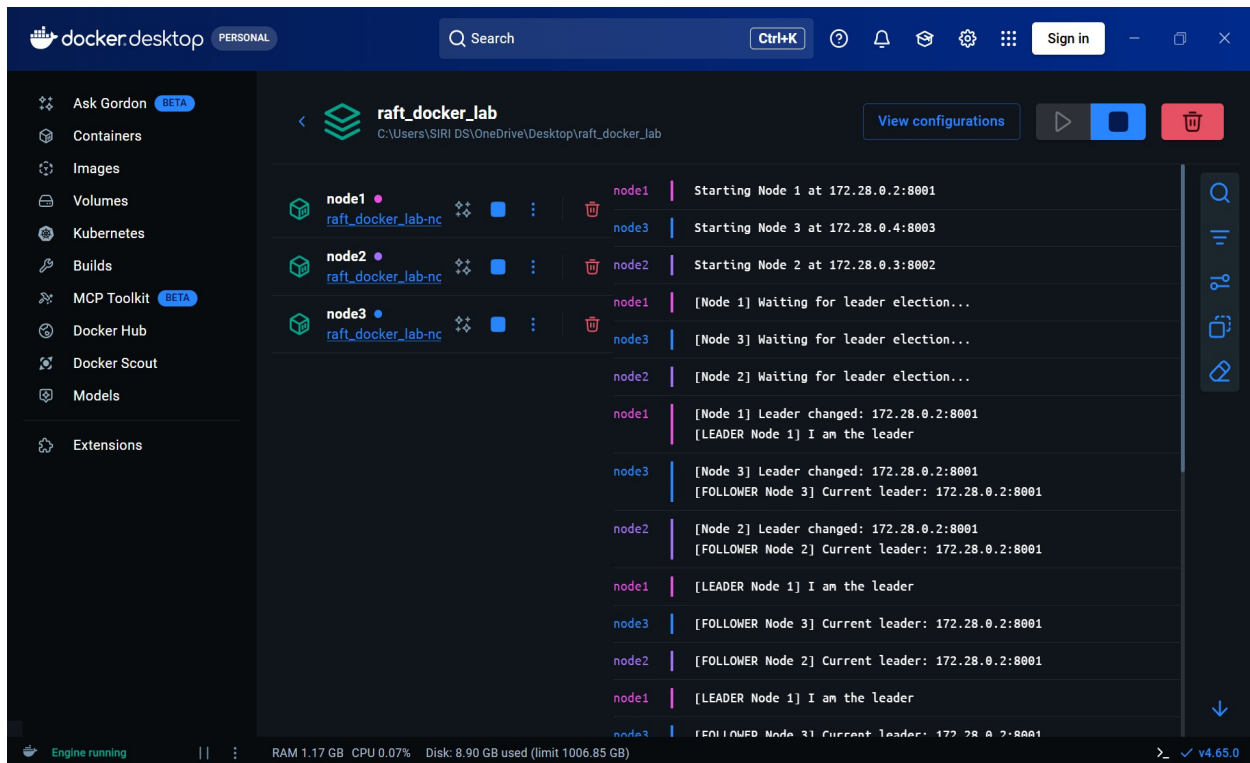
docker compose up --build

Take Screenshot 2A: Docker build process (Terminal)

```
#20 [node3] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 7/7
✓Image raft_docker_lab-node2      Built          4.7s
✓Image raft_docker_lab-node3      Built          4.7s
✓Image raft_docker_lab-node1      Built          4.7s
✓Network raft_docker_lab-raftnet  Created        0.1s
✓Container node3                  Started        0.8s
✓Container node2                  Started        0.8s
✓Container node1                  Started        0.7s

C:\Users\SIRI_DS\OneDrive\Desktop\raft_docker_lab>
```

Take Screenshot 2B: Docker build process (Docker)



12. Observe Leader Election

Observe Leader Election

After starting the containers, students must observe the logs of each node to understand how the Raft algorithm elects a leader.

Step 1: Open Three Separate Terminals

Open three new Command Prompt windows (Terminal 2, Terminal 3, and Terminal 4).

Navigate to the project folder in each terminal:

cd %USERPROFILE%\OneDrive\Desktop\raft_docker_lab

For linux user: cd ~/Desktop/raft_docker_lab

Step 2: Observe Logs of Each Node

Run the following command in Terminal 2:

docker logs -f node1

Run the following command in Terminal 3:

docker logs -f node2

Run the following command in Terminal 4:

```
docker logs -f node3
```

The -f option allows the terminal to continuously display live logs from the container.

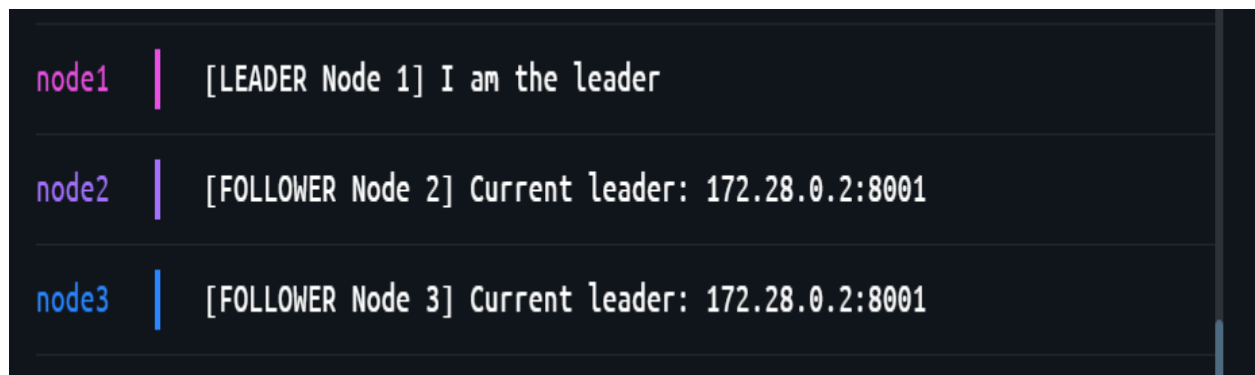
Step 3: Observe Leader Election

Students should observe the outputs appearing in the terminals. One node will become the Leader, while the other nodes will act as Followers.

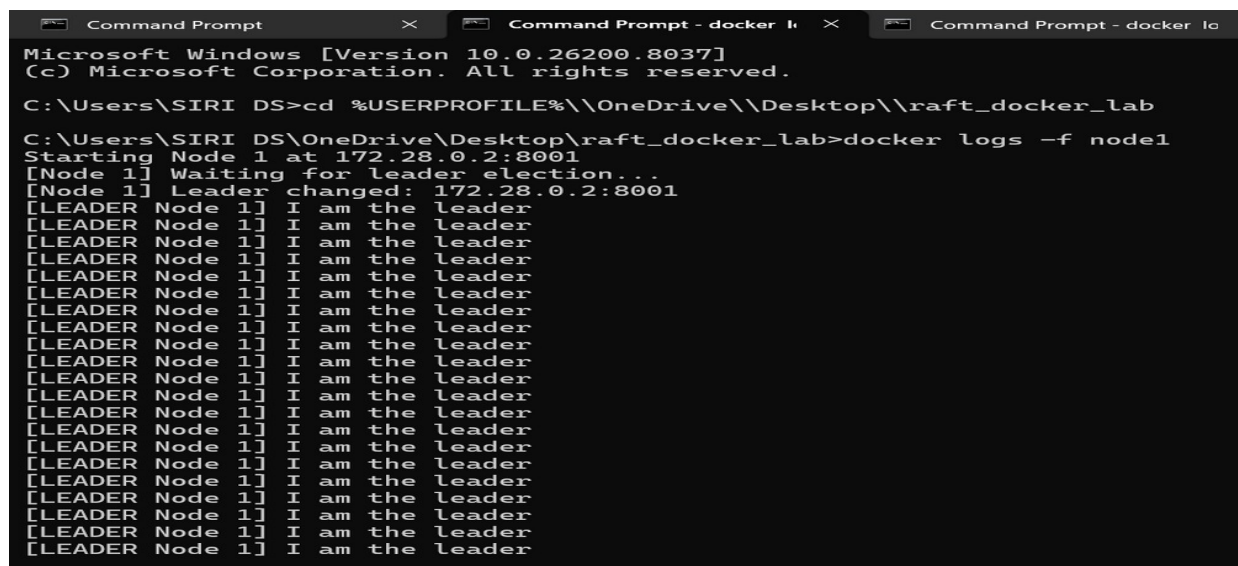
Step 4: Observe Logs in Docker Desktop

Students can also observe the same logs through the Docker Desktop interface by selecting the running containers node1, node2, and node3 and viewing their logs

Take Screenshot 3A: Leader election logs from docker



Take Screenshot 3B: Leader election logs from Terminal 1



Take Screenshot 3C: Leader election logs from Terminal 2

[illegible]

Take Screenshot 3D: Leader election logs from Terminal 3

[illegible]

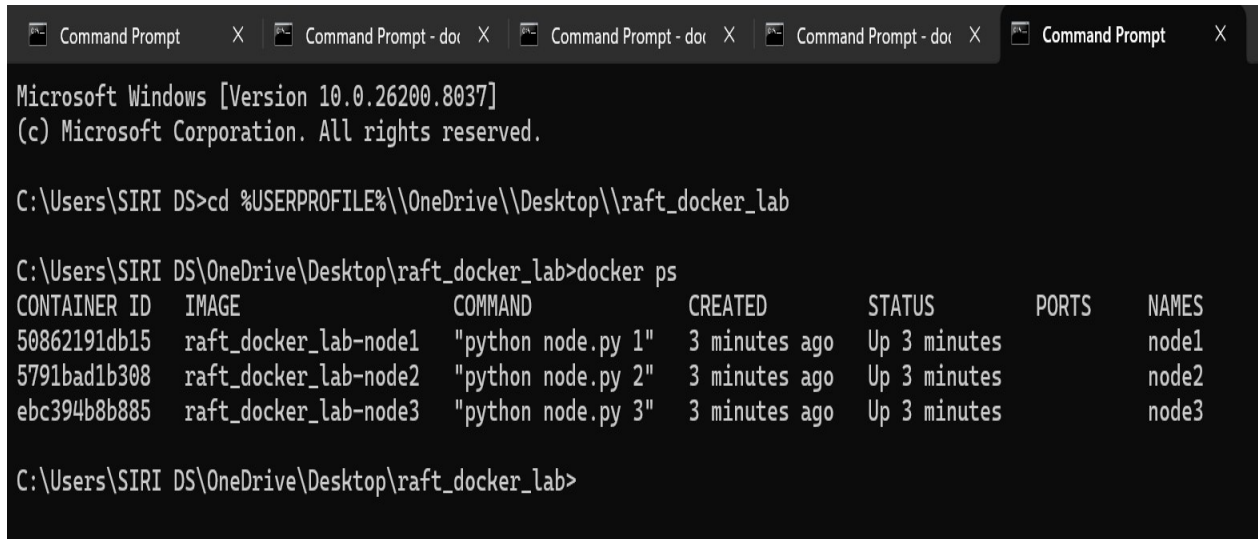
13. Check Running Containers

Open another terminal

Run:

docker ps

Take Screenshot 4: Running containers



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SIRI DS>cd %USERPROFILE%\OneDrive\Desktop\raft_docker_lab

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS        NAMES
50862191db15   raft_docker_lab-node1 "python node.py 1"      3 minutes ago Up 3 minutes             node1
5791bad1b308   raft_docker_lab-node2 "python node.py 2"      3 minutes ago Up 3 minutes             node2
ebc394b8b885   raft_docker_lab-node3 "python node.py 3"      3 minutes ago Up 3 minutes             node3

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>
```

14. Check Docker Network

Run:

docker network inspect raft_docker_lab_raftnet

Take Screenshot 5: Docker network configuration


```
Command Prompt  X  Command Prompt - doc  X  Command Prompt - doc  X  Con

✓Container node1          Started

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker network inspect raft_docker_lab_raftnet
[
  {
    "Name": "raft_docker_lab_raftnet",
    "Id": "f1222a34cd89fa04f4bb28478d5d93e39085bedc01ebeeef70b2864b340985d",
    "Created": "2026-03-16T17:50:54.92932956Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.28.0.0/16",
          "Gateway": "172.28.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {
      "com.docker.compose.config-hash": "7b955128463677559d4edba13e9fe8f69dd803b2dfc190dd0f2333946fc6a5d0",
      "com.docker.compose.network": "raftnet",
      "com.docker.compose.project": "raft_docker_lab",
      "com.docker.compose.version": "5.1.0"
    },
    "Containers": [
      {
        "50862191db154eb1c32cc7f17511643e167457a365c9df24a2b5ef5b558273d2": {
          "Name": "node1",
          "EndpointID": "0654358db2558fe5f71c6e459a2085403aaa87bdeadd42ba8501c0f0503e8097",
          "MacAddress": "ee:1c:f5:de:ce:ec",
          "IPv4Address": "172.28.0.2/16",
          "IPv6Address": ""
        },
      {
        "5791bad1b308b1ba6692f6337de6fb3c9184a1f9301437711979be93b77a4cf3": {
          "Name": "node2",
          "EndpointID": "1b1ac8ccb0d29ab874f0d53e65641d2b545b58009923a7c36bf60a95c15dd884",
          "MacAddress": "e6:21:3f:a0:c7:b1",
          "IPv4Address": "172.28.0.3/16",
          "IPv6Address": ""
        },
      {
        "ebc394b8b88586e51a4f5bf1114633bd201609faacf6294f5dc8f19d8735358f": {
          "Name": "node3",
          "EndpointID": "af8ff21c10642ada019d7d6840570e228f229cec6cb15685a5b414d5ef140666",
          "MacAddress": "ba:19:64:88:a7:5a",
          "IPv4Address": "172.28.0.4/16",
          "IPv6Address": ""
        }
      ]
    },
    "Status": {
      "IPAM": {
        "Subnets": [
          "172.28.0.0/16": {

```

15. Simulate Leader Failure

Run:

docker stop node1

System will elect a new leader.

Take Screenshot 6A: Leader re-election

```
node2 | [Node 2] Leader changed: None  
      | [Node 2] Waiting for leader election...  
node3 | [Node 3] Leader changed: None  
      | [Node 3] Waiting for leader election...  
node2 | [Node 2] Leader changed: 172.28.0.4:8003  
      | [FOLLOWER Node 2] Current leader: 172.28.0.4:8003  
node3 | [Node 3] Leader changed: 172.28.0.4:8003  
      | [LEADER Node 3] I am the leader  
node2 | [FOLLOWER Node 2] Current leader: 172.28.0.4:8003  
node3 | [LEADER Node 3] I am the leader  
node2 | [FOLLOWER Node 2] Current leader: 172.28.0.4:8003  
node3 | [LEADER Node 3] I am the leader
```

Take Screenshot 6B: Leader election logs from Terminal 2

```
[Node 2] Leader changed: None
[Node 2] Waiting for leader election...
[Node 2] Leader changed: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
[FOLLOWER Node 2] Current leader: 172.28.0.4:8003
```

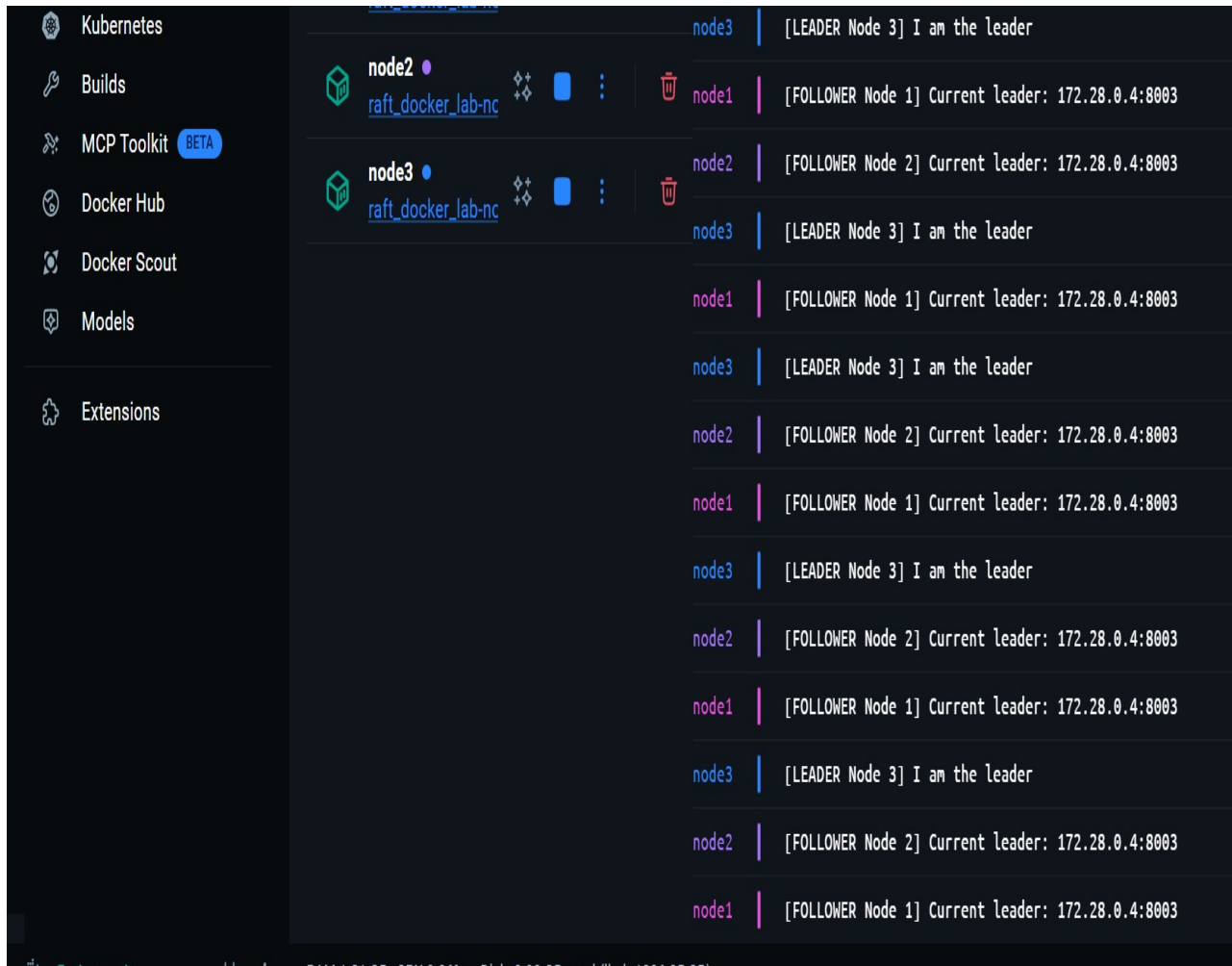
Take Screenshot 6C: Leader election logs from Terminal 3

```
[Node 3] Leader changed: None
[Node 3] Waiting for leader election...
[Node 3] Leader changed: 172.28.0.4:8003
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
[LEADER Node 3] I am the leader
```

16. Restart Node
docker start node1

Node rejoins cluster as follower.

Take Screenshot 7A: Node Rejoining Cluster as Follower from docker



Run: `docker logs -f node1`

Take screenshot 7B: Node Rejoining Cluster as Follower from Terminal 1

```
[LEADER Node 1] I am the leader
Starting Node 1 at 172.28.0.2:8001
[Node 1] Waiting for leader election...
[Node 1] Leader changed: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
[FOLLOWER Node 1] Current leader: 172.28.0.4:8003
```

17. Stop Cluster
docker compose down

Activity:

Students must modify the existing 3-node setup to create a 5-node cluster on their own by updating the configuration files (docker-compose.yml and node.py). After successfully setting up the 5-node cluster, perform the following steps to observe quorum (majority) behavior in Raft.

18. Majority / Quorum Experiment using 5-Node Cluster

Open another terminal

Run:

docker compose up --build

Take Screenshot 10:

[illegible]

```
Run:docker logs -f node3
```

Take Screenshot 11:

[illegible]

```
Run:docker logs -f node4
```

Take Screenshot 12:

[illegible]

Run: docker logs -f node5

Take Screenshot 13:

```
C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker logs -f node5
Starting Node 5 at 172.28.0.6:8005
[Node 5] No leader elected yet
[Node 5] Leader changed: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
[FOLLOWER Node 5] Current leader: 172.28.0.2:8001
```

Observe that:

- one node becomes leader
- other nodes act as followers

Observation: Leader election in 5-node cluster

Stop Two Nodes

Open another terminal

Run:

docker stop node4

docker stop node5

Check Running Containers After Stopping Two Nodes

Open another terminal

Run:

docker ps

Verify that only these containers are running:

- node1
- node2
- node3

Take Screenshot 14: Cluster running after stopping node4 and node5


```
C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker stop node4
node4

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker stop node5
node5

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d61948e3960f	raft_docker_lab-node1	"python node.py 1"	3 minutes ago	Up 3 minutes		node1
f7ece4e2cdd2	raft_docker_lab-node2	"python node.py 2"	3 minutes ago	Up 3 minutes		node2
3903be260355	raft_docker_lab-node3	"python node.py 3"	3 minutes ago	Up 3 minutes		node3

Observe Logs of Remaining Nodes

In separate terminals, run:

`docker logs -f node1`

`docker logs -f node2`

`docker logs -f node3`

Observe that the cluster is still active and one node is leader.

Cluster still working with 3 nodes

Stop One More Node

Open another terminal

Run:

`docker stop node3`

Check Running Containers After Breaking Majority

Open another terminal

Run:

`docker ps`

Verify that only these containers are running:

- node1
- node2

Take Screenshot 15: Only 2 nodes running after stopping node3

```
C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker stop node3
node3

C:\Users\SIRI DS\OneDrive\Desktop\raft_docker_lab>docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d61948e3960f	raft_docker_lab-node1	"python node.py 1"	5 minutes ago	Up 5 minutes		node1
f7ece4e2cdd2	raft_docker_lab-node2	"python node.py 2"	5 minutes ago	Up 5 minutes		node2

Observe Logs After Majority is Broken

Open separate terminals

Run:

docker logs -f node1

Take Screenshot 16:

```
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[LEADER Node 1] I am the leader
[Node 1] Leader changed: None
[Node 1] No leader elected yet
[Node 1] No leader elected yet
[Node 1] No leader elected yet
[Node 1] No leader elected yet
[Node 1] No leader elected yet
[Node 1] No leader elected yet
```

Run:docker logs -f node2

Take Screenshot 17:

```
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[FOLLOWER Node 2] Current leader: 172.28.0.2:8001
[Node 2] Leader changed: None
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
[Node 2] No leader elected yet
```

Observe the output.

Logs after majority is broken

Stop the Cluster

Run:

`docker compose down`

19.Q&A

Why did leader election not occur when only 2 nodes were active in a 5-node cluster?

20. Result

The Raft distributed cluster was successfully implemented using Docker containers. The system performed leader election and demonstrated fault tolerance when the leader node failed. The experiment was extended to a 5-node cluster to observe quorum behavior. The system continued to function when two nodes were stopped, but became unavailable when three nodes were stopped, proving the majority requirement in Raft.

21. Conclusion

This experiment demonstrates how Docker can be used to simulate distributed systems and how the Raft consensus algorithm elects a leader and maintains system consistency. It also shows the importance of quorum (majority), where the system continues to operate only when the majority of nodes are available. When the majority is lost, the system becomes unavailable, ensuring consistency and preventing incorrect decisions.